ktunotes.in

GET IT ON Google Play



KTU NOTES

The learning companion.

KTU STUDY MATERIALS | SYLLABUS | LIVE NOTIFICATIONS | SOLVED QUESTION PAPERS

Website: www.ktunotes.in

CST414
DEEP LEARNING
Module-5 PART -I

Ktunotes.in

SYLLABUS

- **Module-5 (Application Areas)**
- **Applications – computer vision, speech recognition, natural language processing, common word embedding**: continuous Bag-of-Words, Word2Vec, global vectors for word representation (GloVe). Research Areas – autoencoders, representation learning, boltzmann machines, deep belief networks.

KTUQuotes.in

Applications

- solve applications in computer vision, speech recognition, natural language processing

Computer Vision

- Applications of computer vision range from reproducing human **visual abilities, such as recognizing faces,** to creating entirely new categories of visual abilities.
- **one recent computer vision application is to recognize sound waves from the vibrations they induce in objects visible in a video**
- Most deep learning for computer vision is used for object recognition or detection of some form, whether this means reporting which object is present in an image

KRACLER.in

- Annotating an **image with bounding boxes around each object**, transcribing a sequence of symbols from an image, or labeling each pixel in an image with the identity of the object it belongs to.
- Deep learning models capable of image synthesis are usually useful for **image restoration**, a computer vision task involving **repairing defects** in images or **removing objects from images**.

Preprocessing:-

- The images should be standardized so that **their pixels** all lie in the same, **reasonable range, like [0,1] or [-1, 1].**
- Formatting images to have the **same scale** is the only kind of preprocessing that is strictly necessary.
- Many computer vision architectures require images of a standard size, so images must **be cropped or scaled to fit that size.**

- **Dataset augmentation** may be seen as a way of preprocessing the training set only.
- Dataset augmentation is an excellent way **to reduce the generalization error** of most computer vision models
- A related idea applicable **at test time** is to show the model many different versions of the same input (for example, the same image cropped at slightly different locations) and have the **different instantiations** of the model vote to determine the output.
- This latter idea can be interpreted as an **ensemble approach**, and **helps to reduce generalization error**.
- Other kinds of preprocessing are applied to **both the train and the test set with the goal of putting each example** into a more canonical form in order to reduce the amount of variation that the model needs to account for.

KRAnotes.in

- both reduce generalization error and reduce the size of the model needed to fit the training set
- ***Contrast Normalization***
- One of the most obvious sources of variation that can be safely removed for many tasks is the amount **of contrast in the image**
- Contrast simply refers to the **magnitude of the difference between the bright and the dark pixels** in an image.
- In the context of deep learning, contrast usually refers to the **standard deviation of the pixels in an image** or region of an image.

RAMCES.in

- Suppose we have an image represented by a tensor $\mathbf{X} \in \mathrm{R}r \times c \times 3$, with $Xi,j,1$ being the red intensity at row i and column j , $Xi,j,2$ giving the green intensity and $Xi,j,3$ giving the blue intensity.
- Then the contrast of the entire image is given by

$$\sqrt{\frac{1}{3rc}\sum_{i=1}^{r}\sum_{j=1}^{c}\sum_{k=1}^{3}(X_{i,j,k}-\bar{\mathbf{X}})^2} \qquad (12.1)$$

where $\bar{\mathbf{X}}$ is the mean intensity of the entire image:

$$\bar{\mathbf{X}} = \frac{1}{3rc}\sum_{i=1}^{r}\sum_{j=1}^{c}\sum_{k=1}^{3}X_{i,j,k}. \qquad (12.2)$$

Kraotes.in KRAOTES

- **Global contrast normalization** (GCN) aims to prevent images from having varying amounts of contrast by subtracting the mean from each image, then rescaling it so that the standard deviation across its pixels is equal to some constant s.

- **no scaling factor** can change the contrast of a **zero-contrast** image (one whose pixels all have equal intensity). Images with very low but non-zero contrast often have little information content.
- Introducing a small, positive regularization parameter λ to bias the estimate of the standard deviation.
- One can constrain the denominator to be at least ε . given an input image **x**, gcn produces an output image **x'**, defined such that

$$X'_{i,j,k} = s \frac{X_{i,j,k} - \bar{X}}{\max\left\{\epsilon, \sqrt{\lambda + \frac{1}{3rc}\sum_{i=1}^{r}\sum_{j=1}^{c}\sum_{k=1}^{3}\left(X_{i,j,k} - \bar{X}\right)^2}\right\}}. \qquad (12.3)$$

- Small images cropped randomly are more likely to have nearly constant intensity, making aggressive regularization more useful..

- The scale parameter s can usually be set to 1, as done by **Coates *et al.* (2011**),or chosen **to make each individual pixel have standard deviation across examples close to 1**,
- The standard deviation in equation 12.3 is just a rescaling of the L2 norm of the image (assuming the mean of the image has already been removed).
- It is preferable **to define GCN in terms of standard deviation rather than L2 norm because the standard deviation includes division by the number of pixels**, so GCN based on standard deviation allows the same s to be used regardless of image size
- One can understand GCN **as mapping examples to a spherical shell**
- GCN avoids these problems by reducing each example to a direction rather than a direction and a distance

- There is a preprocessing operation known as **sphering** and it is not the same operation as GCN.
- Sphering does not refer to making the data lie on a spherical shell, but rather to rescaling the principal components to have equal variance, so that the multivariate normal distribution used by PCA has spherical contours.
- Sphering is more commonly known as **whitening**
- **Global contrast normalization will often fail to highlight image features we would like to stand out, such as edges and corners**

krauces.in

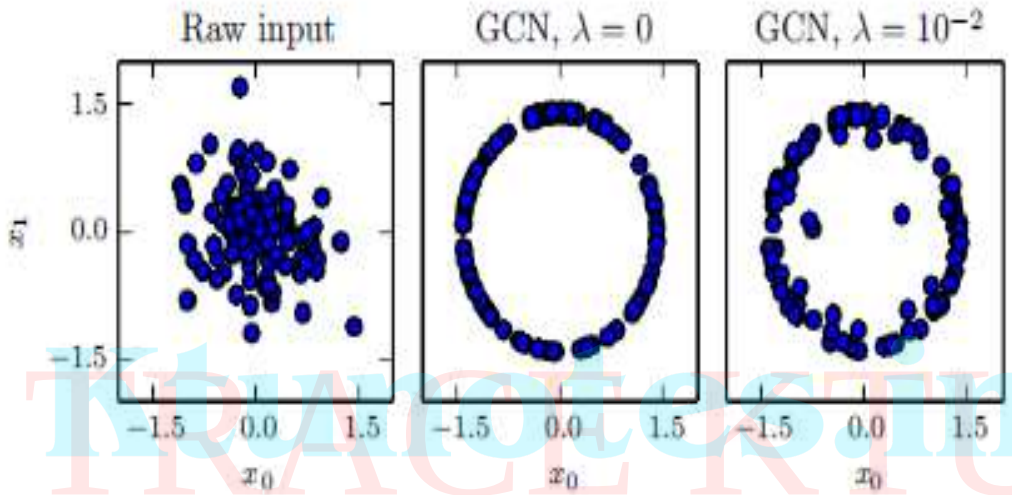


Figure 12.1: GCN maps examples onto a sphere. *(Left)*Raw input data may have any norm. *(Center)*GCN with $\lambda = 0$ maps all non-zero examples perfectly onto a sphere. Here we use $s = 1$ and $\epsilon = 10^{-8}$. Because we use GCN based on normalizing the standard deviation rather than the L^2 norm, the resulting sphere is not the unit sphere. *(Right)*Regularized GCN, with $\lambda > 0$, draws examples toward the sphere but does not completely discard the variation in their norm. We leave s and ϵ the same as before.

- **Local contrast normalization** ensures that the contrast is **normalized across each small window**, rather than over the image as a whole.
- In all cases, **one modifies each pixel by subtracting a mean of nearby pixels and dividing by a standard deviation of nearby pixels.**
- In some cases, this is literally the mean and standard deviation of **all pixels in a rectangular window centered** on the pixel to be modified (Pinto *et al.*, 2008).
- In other cases, this is a **weighted mean and weighted standard deviation using Gaussian weights** centered on the pixel to be modified.

KRACKER.IO

- Local contrast normalization can usually be implemented **efficiently by using separable convolution** to compute feature maps of local means and local standard deviations, then using **element-wise subtraction and element-wise division on different feature maps.**
- **Local contrast normalization is a differentiable** operation and can also be used as a nonlinearity applied to the hidden layers of a network, as well as a preprocessing operation applied to the input.

KTUCLASS.in

Dataset Augmentation

- It is easy to improve the generalization of a classifier by **increasing the size of the training set by adding extra copies of the training examples** that have been modified with transformations that do not change the class.
- **Object recognition** is a classification task that is especially amenable to this form of dataset augmentation because the **class is invariant to so many transformations and the input can be easily transformed with many geometric operations**.
- In specialized computer vision applications, more advanced transformations are commonly used for dataset augmentation. These schemes include random perturbation of the colors in an image (Krizhevsky *et al.*, 2012) and nonlinear geometric distortions of the input (LeCun *et al.*, 1998b).

Speech Recognition

- The task of speech recognition is **to map an acoustic signal containing a spoken natural language utterance into the corresponding sequence of words** intended by the speaker
- Let $X = (x(1), x(2), \ldots, x(T))$ denote the sequence of acoustic input vectors (traditionally produced by splitting the audio into **20ms frames**).
- Most speech recognition systems preprocess the input using specialized hand-designed features,
- Let $y = (y1, y2, \ldots, yN)$ denote the target output sequence (usually a sequence of words or characters). The **automatic speech recognition** (ASR) task consists of creating a function $f*ASR$ that computes the most probable linguistic sequence y given the acoustic sequence X:

$$f^*_{\text{ASR}}(\boldsymbol{X}) = \arg\max_{\boldsymbol{y}} P^*(\mathrm{y} \mid \mathbf{X} = \boldsymbol{X}) \qquad (12.4)$$

- where ***P*** $*$ **is the true conditional distribution relating the inputs *X* to the targets *y*.**
- **To solve speech recognition tasks, unsupervised pretraining was used to build deep feedforward networks whose layers were each initialized by training an RBM**.
- These networks take spectral acoustic representations in a fixed-size input window (around a center frame) and predict the conditional probabilities of HMM states for that center frame.

Khanoteskin

- Another important push, still ongoing, has been towards end-to-end deep learning speech recognition systems that completely remove **the HMM**.
- The first major breakthrough in this direction came from Graves *et al.* (2013) who trained a deep LSTM RNN , using MAP inference over the frame-to phoneme alignment, as in LeCun *et al.* (1998b) and in the CTC framework (Graves *et al.*, 2006; Graves, 2012)
- Another contemporary step toward end-to-end deep learning ASR is to let the system learn how to “align” the acoustic-level information with the phonetic-level Information

Natural Language Processing

- **Natural language processing** (NLP) is the use of human languages, such as English or French, by a computer
- Natural language processing includes applications such as **machine translation, in which the learner must read a sentence in one human language and emit an equivalent sentence** in another human language
- Many NLP applications are **based on language models** that define **a probability distribution over sequences of words**, characters or bytes in a natural language.
- To build an efficient model of natural language, we must usually use techniques that are specialized for **processing sequential data**. In many cases, we choose to regard natural language as a **sequence of words**, rather than a sequence of individual characters or bytes

***n*-grams**

- **A language model defines a probability distribution over sequences of tokens in a natural language**
- Depending on how the model is designed, a token may be a word, a character, or even a byte. **Tokens are always discrete entities**.
- **Language models were based on models of fixed-length sequences of tokens called *n*-grams**
- An *n*-gram is a sequence of ***n* tokens**.
- **Models based on *n*-grams define the conditional probability of the *n*-th token given the preceding *n* − 1 tokens**

Knuckles.in

- The model uses **products of these conditional distributions** to define the probability distribution over longer sequences

$$P(x_1, \ldots, x_\tau) = P(x_1, \ldots, x_{n-1}) \prod_{t=n}^{\tau} P(x_t \mid x_{t-n+1}, \ldots, x_{t-1}). \qquad (12.5)$$

- The probability distribution over the initial sequence $P(x1, \ldots, xn-1)$ may be modeled by a different model with a smaller value of n.
- Training n-gram models is straightforward because **the maximum likelihood estimate can be computed simply by counting how many times each possible n gram occurs in the training s**et.
- For small values of n, models have particular names: **unigram** for n=1, **bigram** for n=2, and **trigram** for n=3.

KtuNotes.in

- These names derive from the Latin prefixes for the corresponding numbers and the **Greek suffix "-gram" denoting something that is written.**
- Usually we train both an n-gram model and an $n-1$ gram model simultaneously.
- This makes it easy to compute

$$P(x_t \mid x_{t-n+1}, \ldots, x_{t-1}) = \frac{P_n(x_{t-n+1}, \ldots, x_t)}{P_{n-1}(x_{t-n+1}, \ldots, x_{t-1})} \qquad (12.6)$$

- two stored probabilities. For this to exactly reproduce inference in Pn, we must omit the final character from each sequence when we train $P\,n-1$.

- we **demonstrate how a trigram model computes** the probability of the sentence " **THE DOG RAN AWAY**."
- we must use the **marginal probability** over words at the start of the sentence. We thus evaluate *P*3(THE DOG RAN).
- Finally, the last word may be predicted using the typical case, of using the conditional distribution *P*(AWAY | DOG RAN). Putting this together with equation 12.6.
- we obtain:

Krnotes.in

$$P(\text{THE DOG RAN AWAY}) = P_3(\text{THE DOG RAN})P_3(\text{DOG RAN AWAY})/P_2(\text{DOG RAN}). \quad (12.7)$$

- A fundamental limitation of **maximum likelihood for *n*-gram models** is that *Pn* as estimated from training set counts is very likely to be zero in many cases, even though the tuple (*xt*−*n*+1, . . . , *xt*) may appear in the test set.
- When *Pn*−1 is zero, the ratio is undefined, so the model does not even produce a sensible output. When *Pn*−1 is non-zero but *Pn* is zero, the test log-likelihood is −∞. To avoid such catastrophic outcomes, most *n*-gram models employ some form of **smoothing**.
- **Smoothing techniques shift probability mass from the observed tuples to unobserved ones that are similar more likely to avoid counts of zero.**

Ktunotes.in

- One basic technique consists of **adding non-zero probability mass to all of the possible next symbol values**
- Another very popular idea is to form a **mixture model containing higher-order and lower-order *n*-gram models**, with the higher-order models providing more capacity and the lower-order models being
- **Back-off methods** look-up the lower-order *n*-grams if the frequency of the context $xt-1, \ldots, xt-n+1$ is too small to use the higher-order model.
- Classical n-gram models are particularly vulnerable **to the curse of dimensionality.**
- One way to view a classical n-gram model is that it is performing nearest-neighbor lookup

KtuAcademy.in

- The **problem for a language model is even more severe than usual, because any two different words have the same distance from each other in one-hot vector space**
- To overcome these problems, a language model must be able to share knowledge between one word and other semantically similar words.
- To improve the statistical efficiency of *n*-gram models, **class-based language Models** introduce the notion of word categories and then share statistical strength between words that are in the same category.

Ktunotes.in

Neural Language Models

- **Neural language models** or NLMs are a class of language model designed to overcome the curse of dimensionality problem for modeling natural language sequences by using a distributed representation of words.
- **neural language models are able to recognize that two words are similar without losing the ability to encode each word as distinct from the other.**
- Neural language models share statistical strength between one word (and its context) and other similar words and contexts.
- For example, if the word dog and the word cat map to representations that share many attributes, then sentences that contain the word cat can inform the predictions that will be made by the model for sentences that contain the word dog, and vice-versa.

KNACKS.IN

- The **curse of dimensionality requires the model to generalize to a number of sentences that is exponential in the sentence length.**
- The model counters this curse by relating each training sentence to an exponential number of similar sentences.
- **word embeddings**--view the raw symbols as points in a space of dimension equal to the vocabulary size. **The word representations embed those points in a feature space of lower dimension**
- In the original space, every word is represented by a one-hot vector, so **every pair of words is at Euclidean distance √2 from each other**

KRACLE.IO

- In the embedding space, words that frequently appear in similar contexts (or any pair of words sharing some "features" learned by the model) are close to each other. This often results in

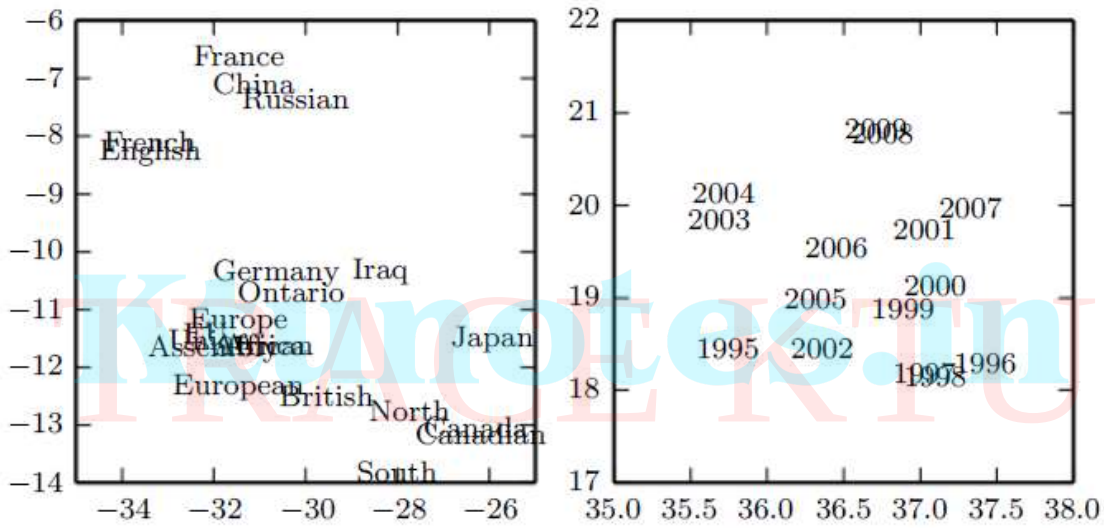


Figure 12.3: Two-dimensional visualizations of word embeddings obtained from a neural machine translation model (Bahdanau *et al.*, 2015), zooming in on specific areas where semantically related words have embedding vectors that are close to each other. Countries appear on the left and numbers on the right. Keep in mind that these embeddings are 2-D for the purpose of visualization. In real applications, embeddings typically have higher dimensionality and can simultaneously capture many kinds of similarity between words.

- on specific areas of a learned **word embedding space to show how semantically similar words map to representations that are close to each other** Neural networks in other domains also define embeddings. For example, a hidden layer of a convolutional network provides an “image embedding.”

TRACE RTU

High-Dimensional Outputs

- we often want our models **to produce words (rather than characters) as the fundamental unit of the output** For large vocabularies, it can be very computationally expensive to represent an output distribution over the choice of a word, because the vocabulary size is large.
- In many applications, V contains hundreds of thousands of words.
- The naive approach to representing such a distribution is to apply an affine transformation from a hidden representation to the output space, then apply the softmax function

KTUCETS.in

- Suppose we have a vocabulary V with size | V | . The weight matrix describing the linear component of this affine transformation is very large, because its output dimension is | V | .
- This imposes a **high memory cost to represent the matrix**, and a high computational cost to multiply by it.
- Because the softmax is normalized across all | V | outputs,
- The **high computational costs of the output layer thus arise both at training time (to compute the likelihood and its gradient) and at test time (to compute probabilities for all or selected words).**

KHOCHER.IO

- Suppose that h is the top hidden layer used to predict the output probabilities $\hat{y}$. If we parametrize the transformation from h to $\hat{y}$ with learned weights W and learned biases b, then the affine-softmax output layer performs the following computations:

$$a_i = b_i + \sum_j W_{ij} h_j \quad \forall i \in \{1, \ldots, |\mathbb{V}|\}, \tag{12.8}$$

$$\hat{y}_i = \frac{e^{a_i}}{\sum_{i'=1}^{|\mathbb{V}|} e^{a_{i'}}}. \tag{12.9}$$

Krnotes.in KRACE R1U

- If h contains nh elements then the above operation is $O(|V| nh)$. With nh in the thousands and $|V|$ in the hundreds of thousands, this operation dominates the computation of most neural language models

Use of a Short List

- The first neural language models (Bengio *et al.*, 2001, 2003) dealt with the high cost of using a softmax over a large number of output words by limiting the vocabulary size to 10,000 or 20,000 words. $\mathbb{T} = \mathbb{V} \backslash \mathbb{L}$
- Schwenk and Gauvain (2002) and Schwenk (2007) built upon this approach by **splitting the vocabulary V into a shortlist L of most frequent words (handled by the neural net) and a tail of more rare words (handled by an *n*-gram model).**
- To be able to combine the two predictions, the neural net also has to predict the probability that a word appearing after context *C* belongs to the tail list.

- This may be achieved by adding an extra sigmoid output unit to provide an estimate of $P(i \in T \mid C)$. The extra output can then be used to achieve an estimate of the probability distribution over all words in V as follows

$$P(y = i \mid C) = 1_{i \in \mathbb{L}} P(y = i \mid C, i \in \mathbb{L})(1 - P(i \in \mathbb{T} \mid C)) + 1_{i \in \mathbb{T}} P(y = i \mid C, i \in \mathbb{T}) P(i \in \mathbb{T} \mid C) \quad (12.10)$$

KRACLEK.in

where $P(y = i \mid C, i \in \mathbb{L})$ is provided by the neural language model and $P(y = i \mid C, i \in \mathbb{T})$ is provided by the n-gram model. With slight modification, this approach can also work using an extra output value in the neural language model's softmax layer, rather than a separate sigmoid unit.

- An obvious disadvantage of the short list approach is that the potential generalization advantage of the neural language models is **limited to the most frequent words, where, arguably, it is the least useful.**
- This disadvantage has stimulated the exploration of alternative methods to deal with high-dimensional outputs,

KRACLES.IN

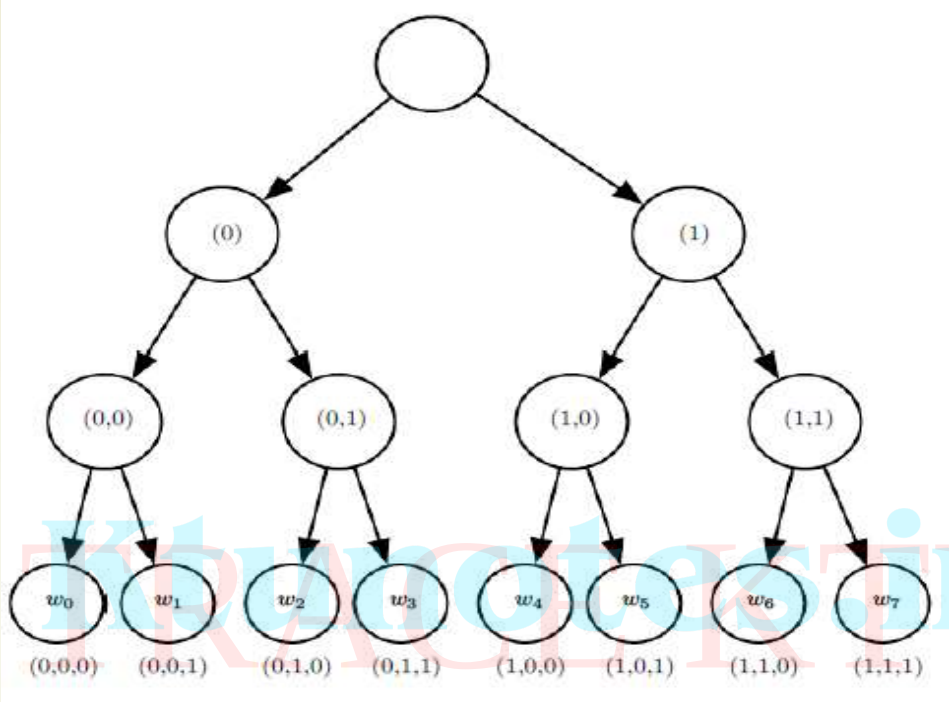
Hierarchical Softmax

- A classical approach (Goodman, 2001) to reducing the computational burden of high-dimensional output layers over large vocabulary sets **V is to decompose probabilities hierarchically.**
- Instead of necessitating a number of computations proportional to $|V|$ (and also proportional to the number of hidden units, nh), **the $|V|$ factor can be reduced to as low as log $|V|$.**
- Bengio (2002) and Morin and Bengio (2005) introduced this **factorized approach** to the context of neural language models.

Kraftes.in

- **hierarchy as building categories of words, then categories of categories of words, then categories of categories of categories of words**, etc.
- These nested categories form a tree, **with words at the leaves**.
- In a balanced tree, **the tree has depth $O(\log |V|)$.**
- The probability of a choosing a word is given by the product of the **probabilities of choosing the branch leading to that word at every node on a path from the root of the tree to the leaf containing the word**.
- Describe how to use multiple paths to identify a single word in order to better model words that have multiple meanings. **Computing the probability of a word then involves summation over all of the paths that lead to that word.**

Ktunotes.in



- Figure 12.4: Illustration of a simple hierarchy of word categories, with 8 words $w0$, . . . , $w7$ organized into a three level hierarchy. The leaves of the tree represent actual specific words. Internal nodes represent groups of words. Any node can be indexed by the sequence of binary decisions (0=left, 1=right) to reach the node from the root.

- Super-class (0) contains the classes (0, 0) and (0, 1), which respectively contain the sets of words $\{w0,w1\}$ and $\{w2,w3\}$, and similarly super-class (1) contains the classes (1,0) and (1, 1), which respectively contain the words ($w4$,$w5$) and ($w6$,$w7$)
- If the tree is sufficiently balanced,**the maximum depth (number of binary decisions) is on the order of the logarithm of the number of words $|V|$** : the choice of one out of $|V|$ words can be obtained by doing $O(\log |V|)$ operations (one for each of the nodes on the path from the root).
- In this example, **computing the probability of a word *y* can be done by multiplying three probabilities**, associated with the binary decisions to move left or right at each node on the path from the root to a node y.

- The probability of sampling an output y decomposes into a product of conditional probabilities, using the chain rule for conditional probabilities, with each node indexed by the prefix of these bits
- For example, node (1, 0) corresponds to the prefix ($b0$ ($w4$) = 1, $b1$ ($w4$) = 0), and the probability of $w4$ can be decomposed as follows:

$$P(\mathrm{y} = w_4) = P(\mathrm{b}_0 = 1, \mathrm{b}_1 = 0, \mathrm{b}_2 = 0) \tag{12.11}$$

$$= P(\mathrm{b}_0 = 1)P(\mathrm{b}_1 = 0 \mid \mathrm{b}_0 = 1)P(\mathrm{b}_2 = 0 \mid \mathrm{b}_0 = 1, \mathrm{b}_1 = 0). \tag{12.12}$$

Kkrnotes.in

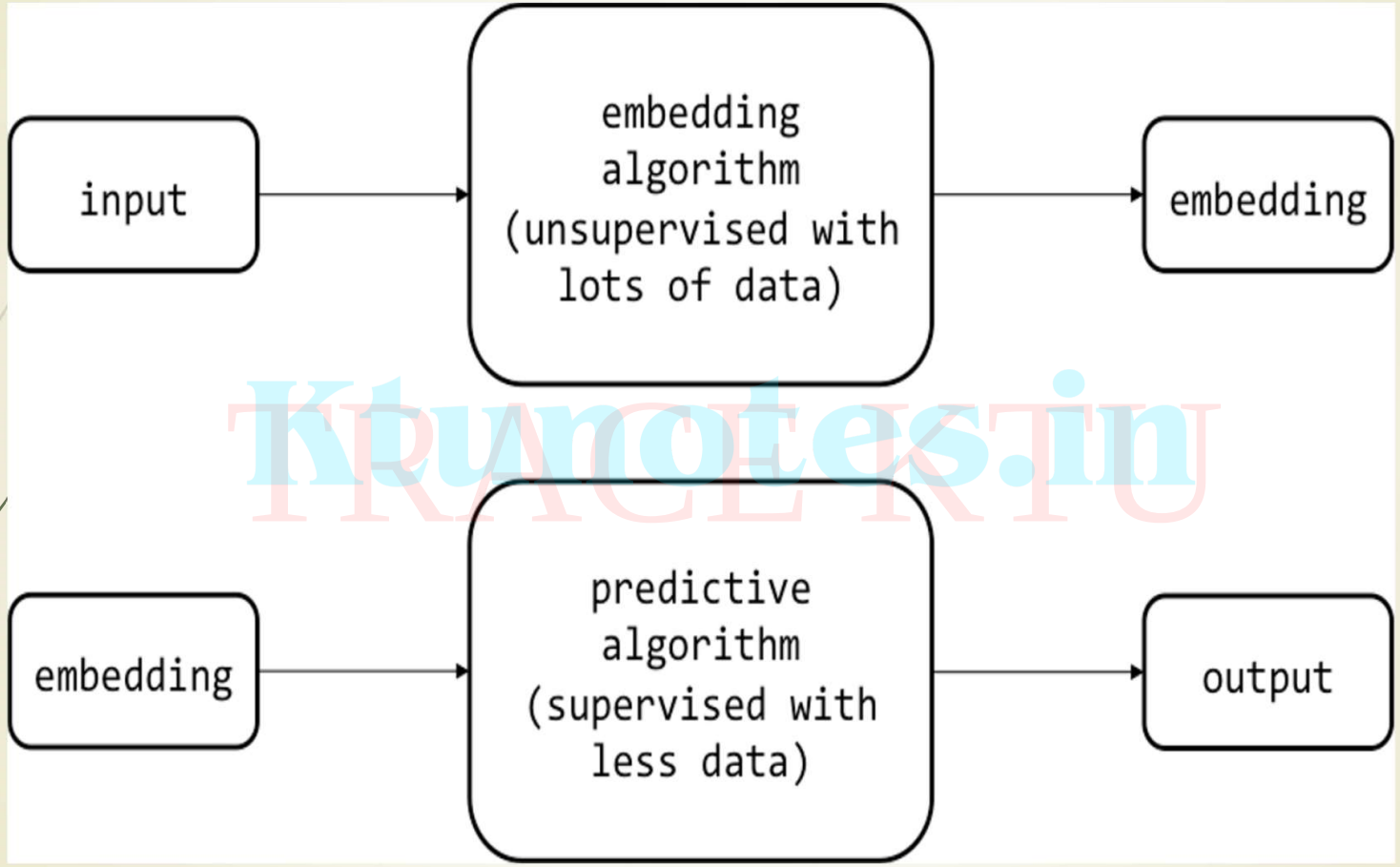
- An important advantage of the hierarchical softmax is that it brings **computational benefits** both at training time and at test time, **if at test time we want to compute the probability of specific words.**
- A disadvantage is that in practice the hierarchical softmax tends to **give worse test results**, This may be due to **a poor choice of word classes.**

TRACE R1U

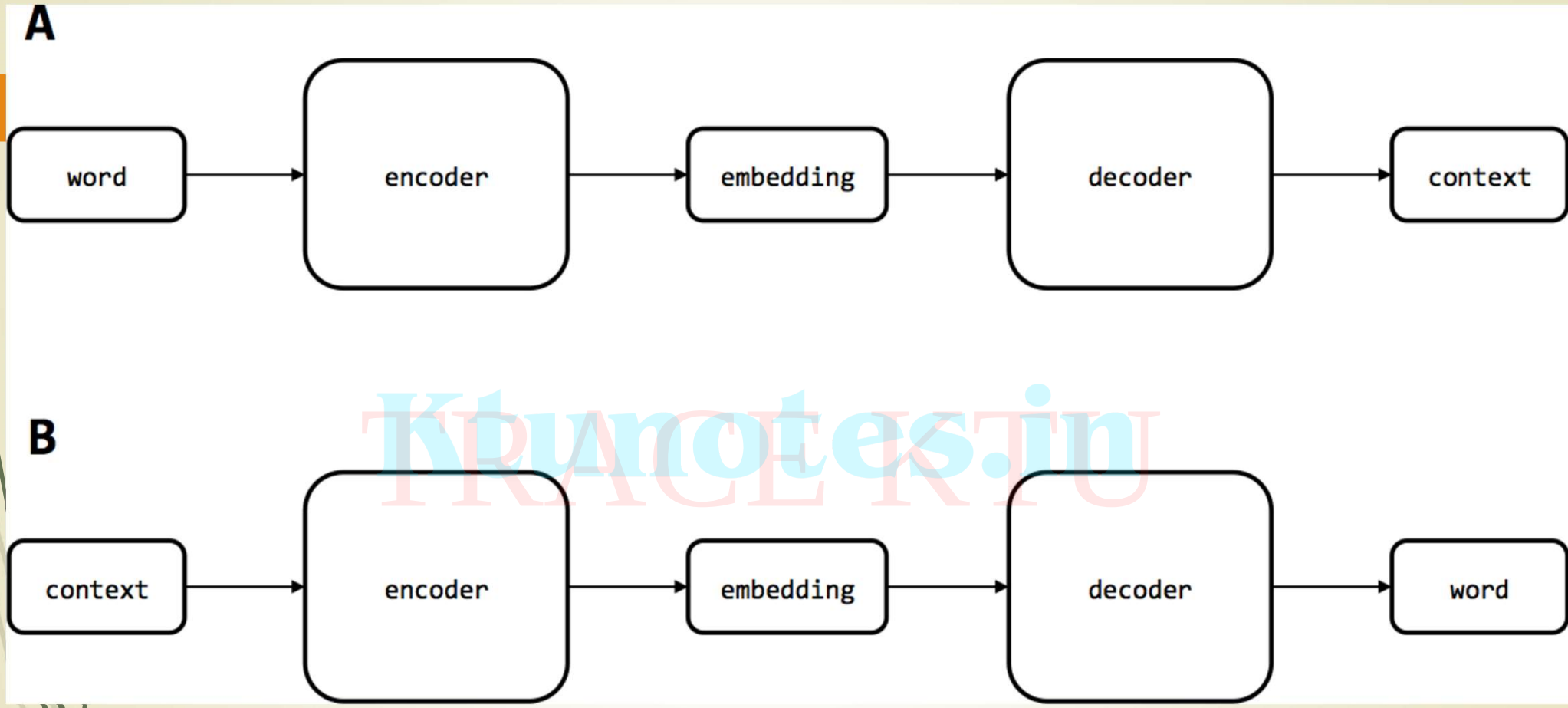
Common Word Embedding

- to develop effective learning models in situations where **labeled data is scarce** but wild, **unlabeled data is plentiful.**
- We'll approach this problem by learning ***embeddings*, or low-dimensional representations,** in an unsupervised fashion.
- Because these unsupervised models allow us to offload all of the heavy lifting of automated feature selection, we can use the generated embeddings to solve learning problems using smaller models that require less data.

Krucles.in

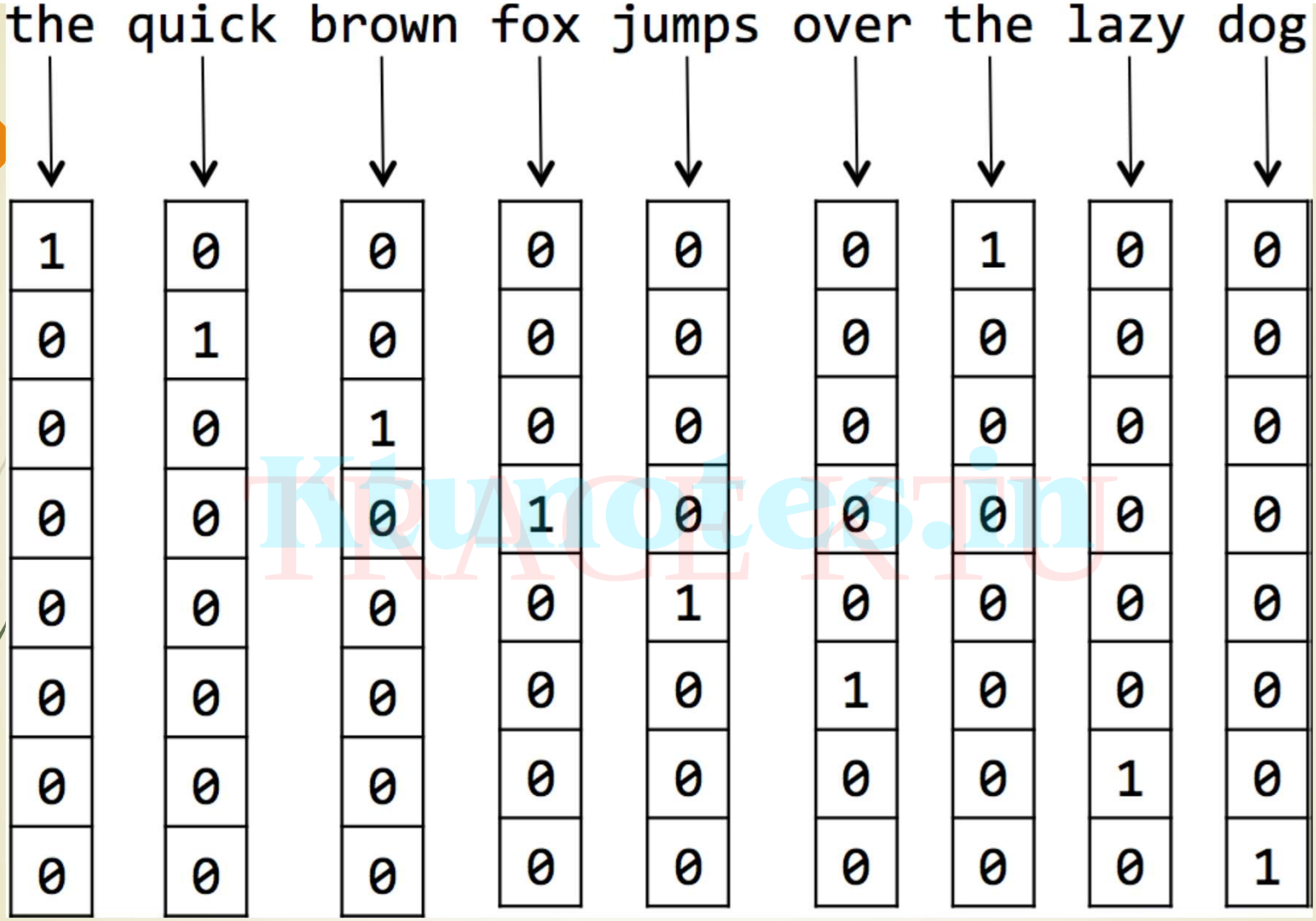

input
embedding algorithm (unsupervised with lots of data)
embedding
embedding
predictive algorithm (supervised with less data)
output

Krucles.in



Krucotes.in

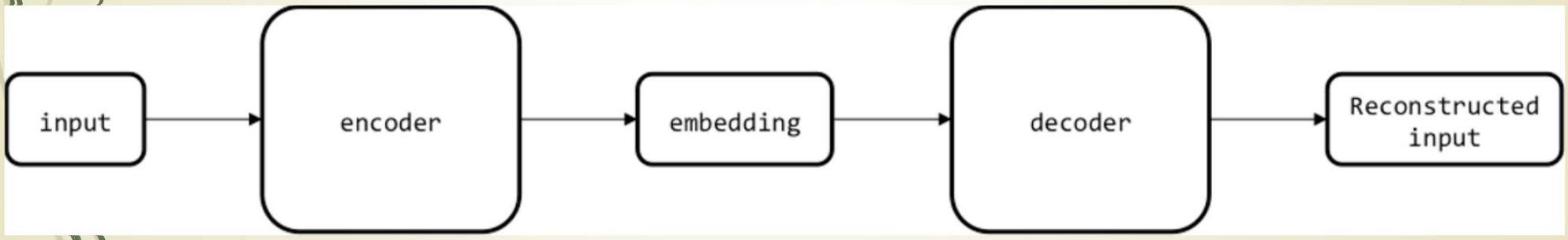
Fig:-General architectures for designing encoders and decoders that generate embeddings by mapping words to their respective contexts (A) or vice versa (B)



Kraces.in

Fig:-An example of generating one-hot vector representations for words using a simple document

- We define a new network architecture that we call the ***autoencoder***. We first take the input and compress it into a low-dimensional vector.
- This part of the network is called the *encoder* because it is responsible for producing the low-dimensional embedding or *code*.
- The second part of the network, instead of mapping the embedding to an arbitrary label as we would in a feed-forward network, tries to invert the computation of the first half of the network and reconstruct the original input. This piece is known as the *decoder*. The overall architecture is illustrated in Figure



CST414
DEEP LEARNING
Module-5 PART -II

Ktunotes.in

SYLLABUS

- **Module-5 (Application Areas)**
- Applications – computer vision, speech recognition, natural language processing, common word embedding: **continuous Bag-of-Words, Word2Vec, global vectors for word representation (GloVe). Research Areas – autoencoders, representation learning, boltzmann machines, deep belief networks.**

KTUCLASS.in

Common Word Embedding

- Word2Vec, a framework for generating word embeddings, was pioneered by Mikolov et al.
- The two variants of word2vec are as follows:
  - 1. **Predicting target words from contexts**: This model tries to predict the ith word, wi, in a sentence using a window of width t around the word. Therefore, the words wi−twi−t+1 ...wi−1wi+1 ...wi+t−1wi+t are used to predict the target word wi. This model is also referred to as the **continuous bag-of-words (CBOW) model**.
  - The CBOW model used the encoder to create an embedding from the full context (treated as one input) and predict the target word. It turns out this strategy works best for smaller datasets

KRACKEK.IO

- 2. **Predicting contexts from target words**: This model tries to predict the context wi−twi−t+1 …wi−1wi+1 …wi+t−1wi+t around word wi, given the ith word in the sentence, denoted by wi. This model is referred to as the **skip-gram model**.
- The Skip-Gram model does the **inverse of CBOW**, taking the target word as an input, and then attempting to predict one of the words in the context

- The first technique is a **multinomial model** which predicts one word out of d outcomes.
- The second model is a **Bernoulli model**, which predicts whether or not each context is present for a particular word. The second model uses negative sampling of contexts for better efficiency and accuracy.

Neural Embedding with Continuous Bag of Words

KRACLES.in

- In the continuous bag-of-words (CBOW) model, the training pairs are all context-word pairs in which **a window of context words is input, and a single target word is predicted**.
- The context contains 2·t words, corresponding to t words both before and after the target word

- we will use the length **m = 2 · t** to define the length of the context.Therefore, **the input to the system is a set of m words**.
- Without loss of generality, let the subscripts of these words be numbered so that they are denoted by w1 …wm, and let the target (output) word in the middle of the context window be denoted by w
- Note that w can be viewed as a categorical variable with d possible values, where d is the size of the lexicon.
- The goal of the neural embedding is **to compute the probability P(w|w1w2 …wm) and maximize the product of these probabilities over all training samples**

Kkanotes.in

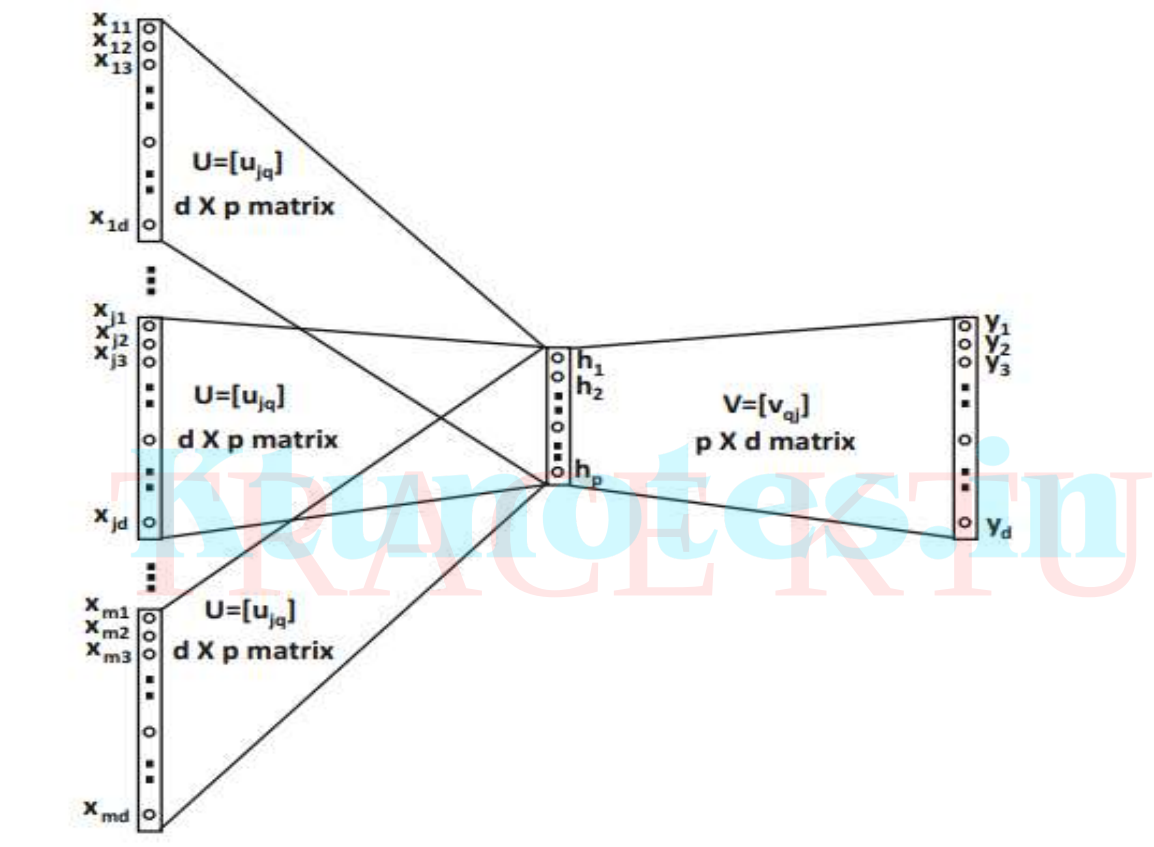


Figure 2.15: Word2vec: The CBOW model

- In the architecture, we have a single input layer with m × d nodes, a hidden layer with p nodes, and an output layer with d nodes.
- The nodes in the input layer are clustered into m different groups, each of which has d units. **Each group with d input units is the one-hot encoded input vector of one of the m context words being modeled by CBOW**.
- Only one of these d inputs will be 1 and the remaining inputs will be 0
- Therefore, **one can represent an input xij with two indices corresponding to contextual position and word identifier.**
- The input xij ∈ {0, 1} contains two indices i and j in the subscript, where i ∈ {1 ...m} is the position of the context, and j ∈ {1 ...d} is the identifier of the word.

Krackeks.in

- The **hidden layer contains p units**, where p is the dimensionality of the hidden layer in word2vec.
- Let h1, h2,...hp be the outputs of the hidden layer nodes
- Note that each of the **d words in the lexicon** has m different representatives in the input layer corresponding to the m different context words, but the weight of each of these m connections is the same.
- **weights are referred to as shared**. Sharing weights is a common trick **used for regularization** in neural networks
- Let the shared weight of each connection from the jth word in the lexicon to the qth hidden layer node be denoted by ujq.
- Note that each of the m groups in the input layer has connections to the hidden layer that are defined by the same d × p weight matrix U

TRACETEST.IN

- uj = (uj1, uj2,...ujp) can be viewed as the p-dimensional embedding of the jth input word over the entire corpus, and
- h = (h1 ...hp) provides the embedding of a specific instantiation of an input context
- Then, **the output of the hidden layer is obtained by averaging the embeddings of the words present in the context.**

Hruoles.in

$$h_q = \sum_{i=1}^{m} \left[\sum_{j=1}^{d} u_{jq} x_{ij} \right] \quad \forall q \in \{1 \ldots p\} \qquad (2.46)$$

$$\overline{h} = \sum_{i=1}^{m} \sum_{j=1}^{d} \overline{u}_j x_{ij} \qquad (2.47)$$

- **The embedding (h1 ...hp) is used to predict the probability that the target word is one of each of the d outputs with the use of the softmax function**. The weights in the output layer are parameterized with a p × d matrix V = [vqj].
- The **jth column of V is denoted by vj .** The output after applying softmax creates d output values **ˆy1 ... yˆd**, which are real values in (0, 1).
- These real values sum to 1 because they can be interpreted as probabilities.
- The ground-truth value of only one of the outputs y1 ...yd is 1 and the remaining values are 0 for a given training instance. One can write this condition as follows:

$$y_j = \begin{cases} 1 & \text{if the target word } w \text{ is the } j\text{th word} \\ 0 & \text{otherwise} \end{cases} \qquad (2.48)$$

Knacks.in

- **The softmax function computes the probability P(w|w1 ...wm) of the one-hot encoded ground-truth outputs** yj as follows:

$$\hat{y}_j = P(y_j = 1|w_1 \ldots w_m) = \frac{\exp(\sum_{q=1}^{p} h_q v_{qj})}{\sum_{k=1}^{d} \exp(\sum_{q=1}^{p} h_q v_{qk})} \qquad (2.49)$$

Kvarotes.in

- For a particular target word w = r ∈ {1 ...d}, the loss function is given by L = −log[P(yr = 1|w1 ...wm)] = −log(ˆyr). **The use of the negative logarithm turns the multiplicative likelihoods over different training instances into an additive loss function using log-likelihoods**

- The updates are defined by using the backpropagation algorithm, as training instances are passed through the neural network one by one
- **loss function can be used to update the gradients of the weight matrix V in the output layer**
- backpropagation can be used to update the weight matrix U between the input and hidden layer. The update equations with learning rate α are as follows:

$$\overline{u}_i \Leftarrow \overline{u}_i - \alpha \frac{\partial L}{\partial \overline{u}_i} \quad \forall i$$

$$\overline{v}_j \Leftarrow \overline{v}_j - \alpha \frac{\partial L}{\partial \overline{v}_j} \quad \forall j$$

- The probability of making a mistake in prediction on the jth word in the lexicon is defined by $|y_j - \hat{y}_j|$. However, we use **signed mistakes** ε_j, in which only the correct word with $y_j = 1$ is given a positive mistake value, while all the other words in the lexicon receive negative mistake values. This is achieved by dropping the modulus:
- $\varepsilon_j = y_j - \hat{y}_j$
- **Note that ε_j can also be shown to be equal to the negative of the derivative of the cross entropy loss with respect to jth input into the softmax layer (which is $h \cdot v_j$)**

Ktunotes.in

- the updates for a particular input context and output word are as follows

$$\overline{u}_i \Leftarrow \overline{u}_i + \alpha \sum_{j=1}^{d} \epsilon_j \overline{v}_j \quad [\forall \text{ words } i \text{ present in context window}]$$

$$\overline{v}_j \Leftarrow \overline{v}_j + \alpha \epsilon_j \overline{h} \quad [\forall j \text{ in lexicon}]$$

- **α > 0 is the learning rate**. Repetitions of the same word i in the context window trigger multiple updates of ui.
- Two different embeddings corresponding to the p-dimensional rows of the matrix U and the p-dimensional columns of the matrix V . The former type of embedding of words is referred to as the **input embedding**, whereas the latter is referred to as the **output embedding**
- . In the CBOW model, the input embedding represents context, and therefore it makes sense to use the output embedding.

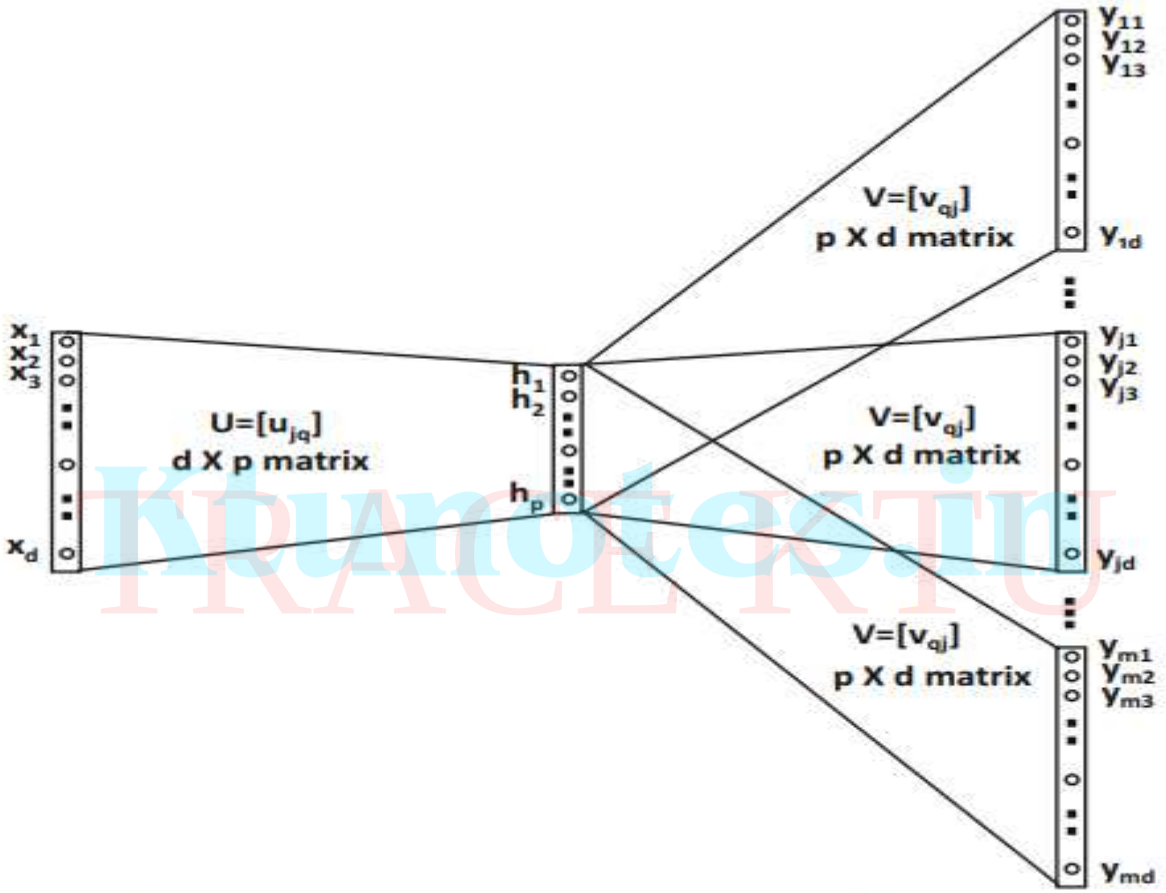
Neural Embedding with Skip-Gram Model

- **In the skip-gram model, the target words are used to predict the m context words. Therefore, we have one input word and m outputs**.
- The skip-gram model is the technique of choice when a large amount of data is available.
- The skip-gram model uses a **single target word w** as the input and **outputs the m context words denoted by w1 ...wm.** Therefore, the goal is to estimate **P(w1, w2....wm|w),** which is different from the quantity P(w|w1 ...wm) estimated in the CBOW model.

Rkmotes.in

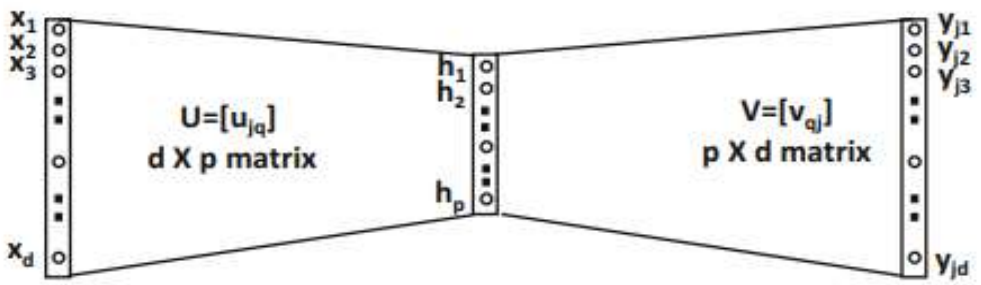
- After such an encoding, the skip-gram model will have **d binary inputs denoted by x1 ...xd** corresponding to the d possible values of the single input word.
- Similarly, the output of each training instance is encoded as m × d values yij ∈ {0, 1}, where i ranges from 1 to m (size of context window), and j ranges from 1 to d (lexicon size
- **Each yij ∈ {0, 1} indicates whether the ith contextual word takes on the jth possible value for that training instance**.
- However, the (i, j)th output node only computes a soft **probability value ˆyij = P(yij = 1|w).**
- Therefore, the probabilities ˆyij in the output layer for fixed i and varying j sum to 1, since the ith contextual position takes on exactly one of the d words. The hidden layer contains p units, the outputs are denoted by h1 ...hp. Each input xj is connected to all the hidden nodes with a d×p matrix U

KTUNotes.in



(a) All elements in context window explicitly shown

Kranoteriq



MINIBATCH THE m d-DIMENSIONAL OUTPUT VECTORS IN EACH CONTEXT WINDOW DURING STOCHASTIC GRADIENT DESCENT. THE SHOWN OUTPUTS y_{jk} CORRESPOND TO THE jth OF m OUTPUTS.

(b) All elements in context window not explicitly shown

KRACKEK.IU

- the p hidden nodes are connected to each of the m groups of d output nodes with the same set of shared weights.
- This set of shared weights between the p hidden nodes and the d output nodes of each of the context words is defined by the p × d matrix V .
- In the case of the skip-gram model, **one can collapse the m identical outputs into a single output, and achieve the same results simply by using a particular type of mini-batching during stochastic gradient descent in fig(a)**

KRNOTES.IN

- all elements of a single context window are always forced to belong to the same mini-batch in **fig (b)**
- The **output of the hidden layer can be computed from the input layer using the d × p matrix of weights U = [ujq]** between the input and hidden layer as follows:

$$h_q = \sum_{j=1}^{d} u_{jq} x_j \quad \forall q \in \{1 \ldots p\} \qquad (2.51)$$

- the one-hot encoding of the input word w in terms of x1 ...xd.
- **If the input word w is the rth word, then one simply copies urq to the qth node of the hidden layer for each q ∈ {1 ...p}.**
- **In other words, the rth row ur of U is copied to the hidden layer.**
- the hidden layer is connected to m groups of d output nodes, each of which is connected to the hidden layer with a p × d matrix V = [vqj].
- Each of these m groups of d output nodes computes the probabilities of the various words for a particular context word.
- The jth column of V is denoted by vj and represents the output embedding of the jth word. The output ˆyij is the probability that the word in the ith context position takes on the jth word of the

Krnotes.in

- the neural network predicts the same **multinomial distribution for each of the context words**. Therefore, we have the following:

$$\hat{y}_{ij} = P(y_{ij} = 1|w) = \underbrace{\frac{\exp(\sum_{q=1}^{p} h_q v_{qj})}{\sum_{k=1}^{d} \exp(\sum_{q=1}^{p} h_q v_{qk})}}_{\text{Independent of context position } i} \quad \forall i \in \{1 \ldots m\} \qquad (2.52)$$

Knowledge.in

- Note that the probability ˆyij is the same for varying i and fixed j, since the right-hand side of the above equation does not depend on the exact location i in the context window

- The loss function for the backpropagation algorithm is the **negative of the log-likelihood values of the ground truth yij ∈ {0, 1} of a training instance.** This loss function L is given by the following:

$$L = -\sum_{i=1}^{m}\sum_{j=1}^{d} y_{ij}\log(\hat{y}_{ij}) \qquad (2.53)$$

Kraotes.in KRACE R1U

- The update equations with learning rate a are as follows:

$$\overline{u}_i \Leftarrow \overline{u}_i - \alpha\frac{\partial L}{\partial \overline{u}_i} \quad \forall i$$

$$\overline{v}_j \Leftarrow \overline{v}_j - \alpha\frac{\partial L}{\partial \overline{v}_j} \quad \forall j$$

The probability of making a mistake in predicting the jth word in the lexicon for the ith context is defined by $|y_{ij} - \hat{y}_{ij}|$. However, we use *signed* mistakes ϵ_{ij} in which only the predicted words (positive examples) have a positive probability. This is achieved by dropping the modulus:

$$\epsilon_{ij} = y_{ij} - \hat{y}_{ij} \tag{2.54}$$

Then, the updates for a particular input word r and its output context are as follows:

$$\overline{u}_r \Leftarrow \overline{u}_r + \alpha \sum_{j=1}^{d} \left[\sum_{i=1}^{m} \epsilon_{ij} \right] \overline{v}_j \quad \text{[Only for input word } r\text{]}$$

$$\overline{v}_j \Leftarrow \overline{v}_j + \alpha \left[\sum_{i=1}^{m} \epsilon_{ij} \right] \overline{h} \quad \text{[For all words } j \text{ in lexicon]}$$

KPAnotes.in

Here, α > 0 is the learning rate. The p-dimensional rows of the matrix U are used as the embeddings of the words. In other words, the convention is to use the input embeddings in the rows of U rather than the output embeddings in the columns of V .

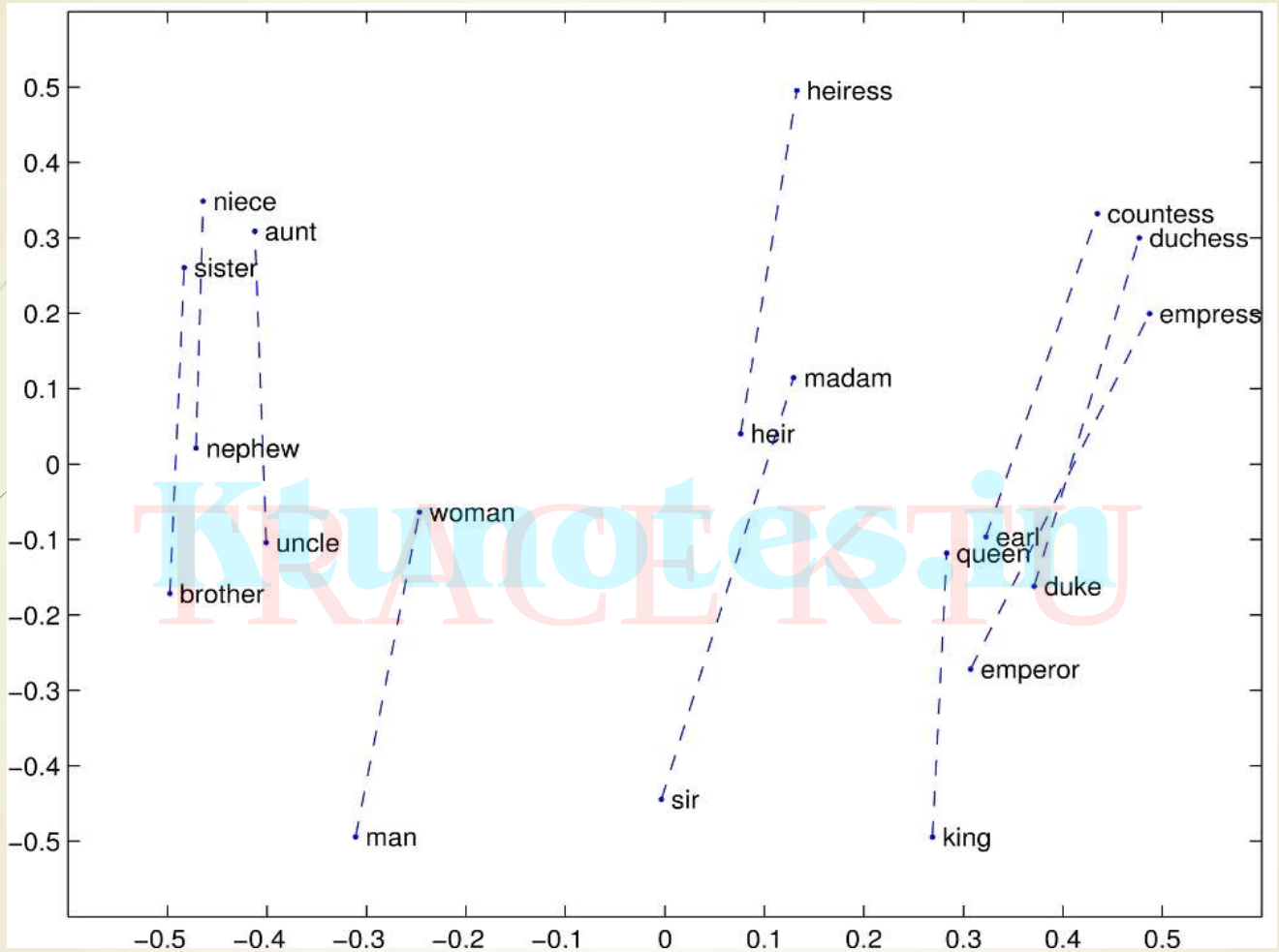
Global Vectors for Word Representation(GloVe)

- GloVe is an unsupervised learning algorithm for obtaining vector representations for words.
- **Jeffrey Pennington, Richard Socher, and Christopher D. Manning. 2014**
- Training is performed on aggregated global word-word co-occurrence statistics from a corpus, and the resulting representations showcase interesting linear substructures of the word vector space.
- Specific weighted least squares model that trains on global word-word co-occurrence counts and thus makes efficient use of statistics.

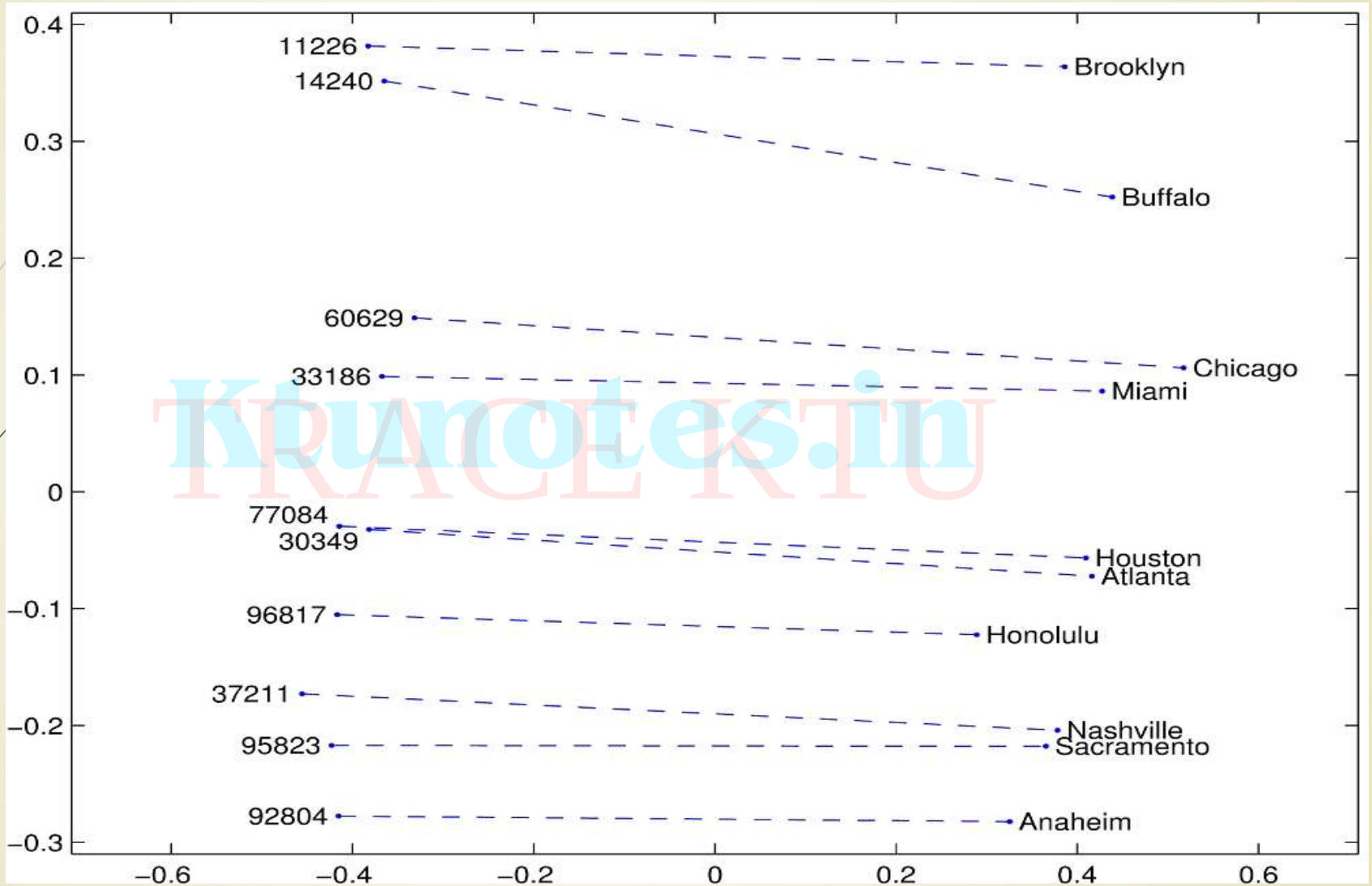
Kuncoteam

- model for word representation which we call GloVe, for Global Vectors, because **the global corpus statistics are captured directly by the model**
- **The Euclidean distance (or cosine similarity) between two word vectors provides an effective method for measuring the linguistic or semantic similarity of the corresponding words**
- The similarity metrics used for **nearest neighbor evaluations** produce a single scalar that quantifies the relatedness of two words. This simplicity can be problematic since two given words almost always exhibit more intricate relationships than can be captured by a single number.
- For example, *man* may be regarded as similar to *woman* in that both words describe human beings

KReCtes.ID



Kraotes.in
KRAOTES.IN


0.4
0.3
0.2
0.1
0
−0.1
−0.2
−0.3
−0.6
−0.4
−0.2
0
0.2
0.4
0.6
11226
14240
Brooklyn
Buffalo
60629
33186
Chicago
Miami
77084
30349
Houston
Atlanta
96817
Honolulu
37211
95823
Nashville
Sacramento
92804
Anaheim

Krucles.in

- The GloVe model is trained on **the non-zero entries** of a **global word-word co-occurrence** matrix, which tabulates how **frequently words co-occur with one another in a given corpus**. Populating this matrix requires a single pass through the entire corpus to collect the statistics.
- For large corpora, this pass can be computationally expensive, but it is a one-time up-front cost. Subsequent training iterations are much faster because the number of non-zero matrix entries is typically much smaller than the total number of words in the corpus.

KRAnotes.in

- Let the matrix of word-word co-occurrence counts be denoted by X, whose entries Xi j tabulate the number of times wPord j occurs in the context of word i.
- Let Xi =∑ k Xik be the number of times any word appears in the context of word i.
- Finally, let Pi j = P(j |i) =Xi j/Xi be the probability that word j appear in the context of word i.

- GloVe is essentially a log-bilinear model with a weighted least-squares objective.
- The main intuition underlying the model is the simple observation that ratios of word-word co-occurrence probabilities have the potential for encoding some form of meaning.

Table 1: Co-occurrence probabilities for target words *ice* and *steam* with selected context words from a 6 billion token corpus. Only in the ratio does noise from non-discriminative words like *water* and *fashion* cancel out, so that large values (much greater than 1) correlate well with properties specific to ice, and small values (much less than 1) correlate well with properties specific of steam.

Probability and Ratio	k = *solid*	k = *gas*	k = *water*	k = *fashion*
$P(k\|ice)$	1.9×10^{-4}	6.6×10^{-5}	3.0×10^{-3}	1.7×10^{-5}
$P(k\|steam)$	2.2×10^{-5}	7.8×10^{-4}	2.2×10^{-3}	1.8×10^{-5}
$P(k\|ice)/P(k\|steam)$	8.9	8.5×10^{-2}	1.36	0.96

- *ice* co-occurs more frequently with *solid* than it does with *gas*, whereas *steam* co-occurs more frequently with *gas* than it does with *solid*.
- Both words co-occur with their shared property *water* frequently, and both co-occur with the unrelated word *fashion* infrequently.
- Only in the ratio of probabilities does noise from non-discriminative words like *water* and *fashion* cancel out, so that large values (much greater than 1) correlate well with properties specific to ice, and small values (much less than 1) correlate well with properties specific of steam.

TRACETER.IN

- Compared to the raw probabilities, the ratio is better able to distinguish relevant words (solid and gas) from irrelevant words (water and fashion) and it is also better able to discriminate between the two relevant words. ratio Pik/Pjk depends on three words i, j, and k, the most general model takes the form,

$$F(w_i, w_j, \tilde{w}_k) = \frac{P_{ik}}{P_{jk}}, \qquad (1)$$

- where w ϵ Rd are word vectors and w˜ ϵ Rd are separate context word vectors.
- The right-hand side is extracted from the corpus, and F may depend on some as-of-yet unspecified parameters.

- functions F that depend only on the difference of the two target words,

$$F(w_i - w_j, \tilde{w}_k) = \frac{P_{ik}}{P_{jk}}. \qquad (2)$$

- we note that the arguments of F in Eqn. (2) are vectors while the right-hand side is a scalar
- While F could be taken to be a complicated function parameterized by, e.g., a neural network, doing so would obfuscate the linear structure we are trying to capture. To avoid this issue, we can first take the dot product of the arguments,

$$F\left((w_i - w_j)^T \tilde{w}_k\right) = \frac{P_{ik}}{P_{jk}}, \qquad (3)$$

Kraftes.in

- note that for word-word co-occurrence matrices, the distinction between a word and a context word is arbitrary
- First, we require that F be a homomorphism between the groups (R,+) and (R>0, ×), i.e.,

$$F\left((w_i - w_j)^T \tilde{w}_k\right) = \frac{F(w_i^T \tilde{w}_k)}{F(w_j^T \tilde{w}_k)}, \quad (4)$$

which, by Eqn. (3), is solved by,

$$F(w_i^T \tilde{w}_k) = P_{ik} = \frac{X_{ik}}{X_i}. \quad (5)$$

The solution to Eqn. (4) is $F = \exp$, or,

$$w_i^T \tilde{w}_k = \log(P_{ik}) = \log(X_{ik}) - \log(X_i). \quad (6)$$

KPUnotes.in

- Next, we note that Eqn. (6) would exhibit the exchange symmetry if not for the log(Xi) on the right-hand side. However, this term is independent of k so it can be absorbed into a bias bi forwi . Finally, adding an additional bias ˜bk for w˜ k restores the symmetry,

$$w_i^T \tilde{w}_k + b_i + \tilde{b}_k = \log(X_{ik}) . \quad (7)$$

- introducing a weighting function f (Xi j) into the cost function gives us the model

$$J = \sum_{i,j=1}^{V} f\left(X_{ij}\right)\left(w_i^T \tilde{w}_j + b_i + \tilde{b}_j - \log X_{ij}\right)^2 , \quad (8)$$

- where V is the size of the vocabulary. The weighting function should obey the following properties:

1. $f(0) = 0$. If f is viewed as a continuous function, it should vanish as $x \to 0$ fast enough that the $\lim_{x \to 0} f(x) \log^2 x$ is finite.
2. $f(x)$ should be non-decreasing so that rare co-occurrences are not overweighted.
3. $f(x)$ should be relatively small for large values of x, so that frequent co-occurrences are not overweighted.

KRACLE.in

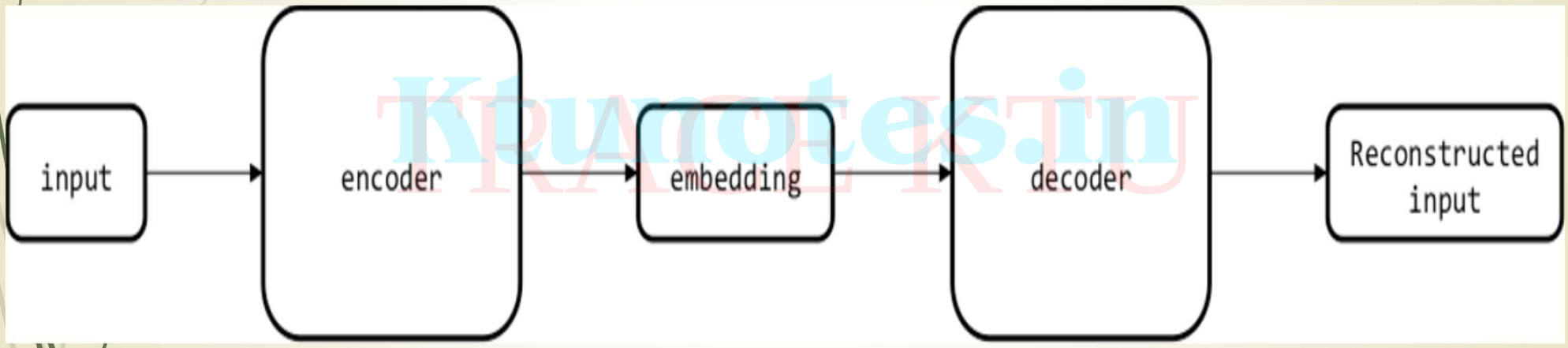
- The basic idea behind the GloVe word embedding is **to derive the relationship between the words from statistics**. Unlike the occurrence matrix, the co-occurrence matrix tells you how often a particular word pair occurs together. Each value in the co-occurrence matrix represents a pair of words occurring together
- The advantage of GloVe is that, unlike Word2vec, **GloVe does not rely just on local statistics (local context information of words), but incorporates global statistics (word co-occurrence) to obtain word vectors**
- **Disadvantages**
  - Because it uses a co-occurrence matrix & global information, memory cost is more in GloVe compared to word2vec.
  - Similar to word2vec, it does not solve the problem of polysemous words since words & vectors have a one-to-one relationship.

RESEARCH AREAS--AUTOENCODERS

- Autoencoders are very useful in the field of **unsupervised machine learning.**
- We can use them to compress the data and reduce its dimensionality.
- An autoencoder is a type of artificial neural network used to **learn efficient codings of unlabeled data** (unsupervised learning).
- An autoencoder learns two functions: **an encoding function** that transforms the input data, and a **decoding function** that recreates the input data from the encoded representation.

KRACLES.in

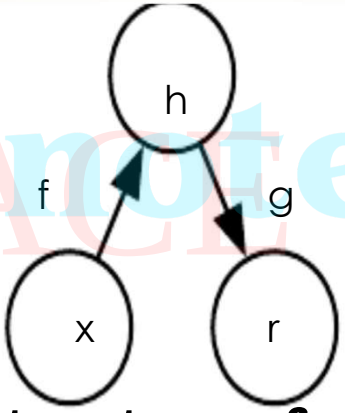
- The main difference between Autoencoders and Principal Component Analysis (PCA) is that while **PCA finds the directions along which you can project the data with maximum** variance, Autoencoders reconstruct our original input given just a compressed version of it.



Krunotes.in

Figure 6-4. The autoencoder architecture attempts to construct a high-dimensional input into a low-dimensional embedding and then uses that low-dimensional embedding to reconstruct the input

- An **autoencoder** is a neural network that is trained to attempt to copy its input to its output it has a hidden layer *h* that describes a **code** used to represent the input.
- The network may be viewed as consisting of two parts: an encoder function $h = f(x)$ and a decoder that produces a reconstruction $r = g(h)$.



Krantes.in

Figure : The general structure of an autoencoder, mapping an input *x* to an output (called reconstruction) *r* through an internal representation or code *h*. The autoencoder has two components: the encoder *f* (mapping *x* to *h*) and the decoder *g* (mapping *h* to *r*).

- Modern autoencoders have generalized the idea of an encoder and a decoder beyond deterministic functions to stochastic mappings *P*encoder(*h* | *x*) and *p*decoder(*x* | *h*).
- **autoencoders were used for dimensionality reduction or feature learning.**

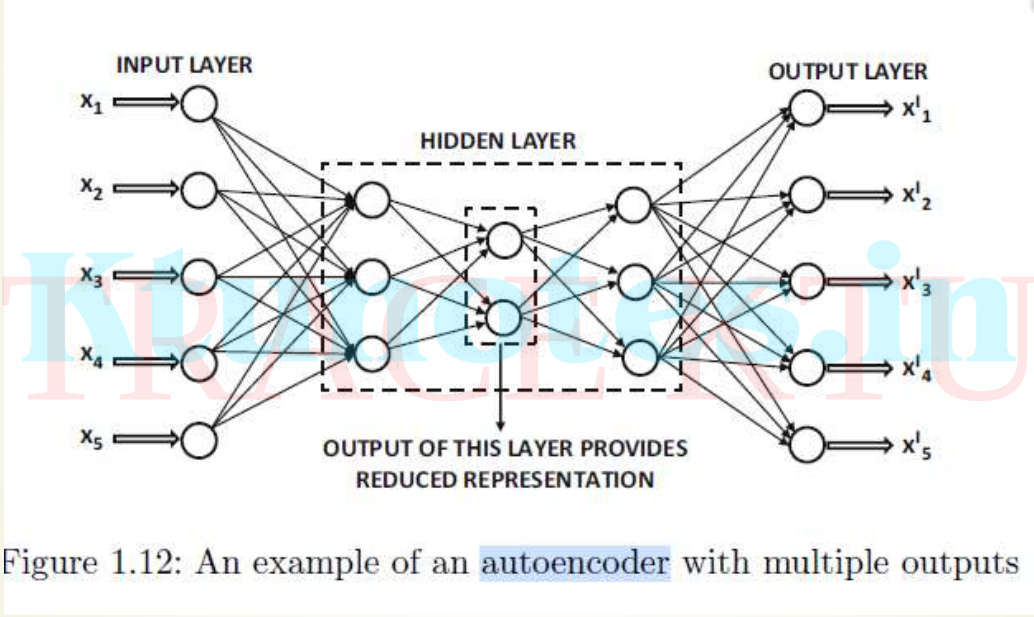


Figure 1.12: An example of an autoencoder with multiple outputs

- A classical example of the dimensionality reduction setting is the autoencoder, which recreates the outputs from the inputs. Therefore, the number of outputs and inputs is equal.
- The constricted **hidden layer in the middle outputs the reduced representation of each instance.**
- As a result of this constriction, there is some loss in the representation, which typically corresponds to the **noise in** the data.
- The outputs of the hidden layers correspond to the reduced representation of the data

KRACKEN.IO

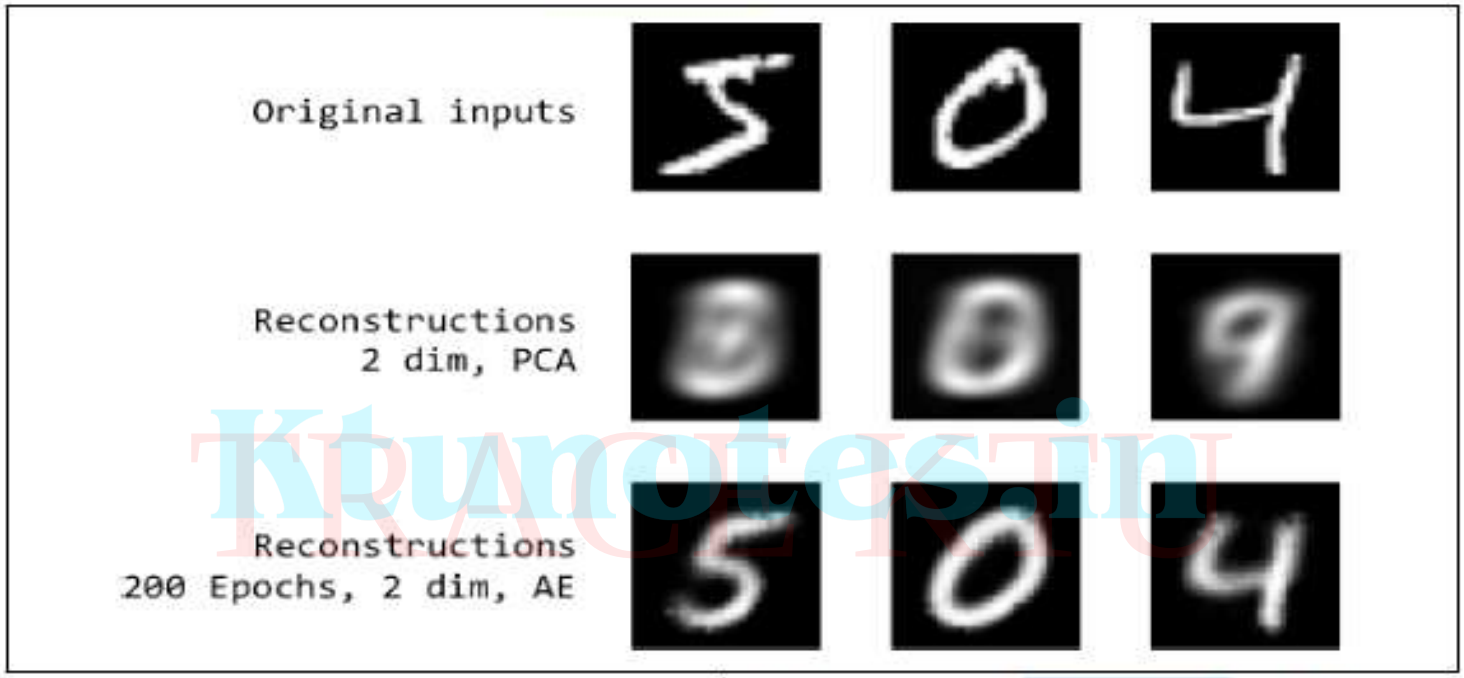


Figure 6-10. Comparing the reconstructions by both PCA and autoencoder side by side

- An Autoencoder is a type of neural network that can learn to reconstruct images, text, and other data from compressed versions of themselves.
- An Autoencoder consists of three layers:

1. Encoder
2. Code
3. Decoder

- The **Encoder layer compresses the input image into a latent space representation**. It encodes the input image as a compressed representation in a reduced dimension.
- The compressed image is a distorted version of the original image.
- The Code layer represents the compressed input fed to the decoder layer.

KRACKEK.in

- The decoder layer decodes the encoded image back to the original dimension. The decoded image is reconstructed from latent space representation, and it is reconstructed from the latent space representation and is a lossy reconstruction of the original image.
- ***Training Autoencoders*:**
- Training an autoencoder is unsupervised in the sense that no labeled data is needed.
- **The training process is still based on the optimization of a cost function**.
- The cost function measures the error between the input x and its reconstruction at the output x ^ .
- An autoencoder is composed of an encoder and a decoder.

TRACE KTU

- When you're building an autoencoder, there are a few things to keep in mind.
- First, the code or bottleneck size is the most critical hyperparameter to tune the autoencoder. **It decides how much data has to be compressed**. It can also act as a regularization term.
- Secondly, it's important to remember that the **number of layers is critical when tuning autoencoders. A higher depth increases model complexity, but a lower depth is faster to process.**
- Thirdly, you should pay attention to how many nodes you use per layer. The number of nodes decreases with each subsequent layer in the autoencoder as the input to each layer becomes smaller across the layers.

- An autoencoder whose code dimension is less than the input dimension is called **undercomplete**.
- An autoencoder whose code dimension is more than the input dimension is called **overcomplete**.
- Learning an undercomplete representation forces the autoencoder to capture the most salient features of the training data.
- The learning process is described simply as minimizing a loss function
- $L(x, g(f(x)))$
- where L is a loss function penalizing $g(f(x))$ for being dissimilar from x, such as the mean squared error.

Kvncet.in

- Autoencoders have various use-cases like:
- **Anomaly detection**: autoencoders can identify data anomalies using a loss function that penalizes model complexity. It can be helpful for anomaly detection in financial markets, where you can use it to identify unusual activity and predict market trends.
- **Data denoising image and audio**: autoencoders can help clean up noisy pictures or audio files. You can also use them to remove noise from images or audio recordings.
- **Image inpainting**: autoencoders have been used to fill in gaps in images by learning how to reconstruct missing pixels based on surrounding pixels. For example, if you're trying to restore an old photograph that's missing part of its right side, the autoencoder could learn how to fill in the missing details based on what it knows about the rest of the photo.
- **Information retrieval:** autoencoders can be used as content-based image retrieval systems that allow users to search for images based on their content.

Knowledge.in

RESEARCH AREAS--REPRESENTATION LEARNING

- Representation Learning is concerned with **training machine learning algorithms to learn useful representations**, e.g. those that are interpretable, have latent features, or can be used for transfer learning.
- Learning representations of the data that make it easier to extract useful information when building classifiers or other predictors.
- Feedforward networks trained by supervised learning as performing a kind of representation learning the last layer of the network is typically a linear classifier, such as a softmax regression classifier

KNACLEKIU

- The rest of the network learns to provide a representation to this classifier. Training with a supervised criterion naturally leads to the **representation at every hidden layer** (but more so near the top hidden layer) taking on properties that make the classification task easier
- Representation learning is particularly interesting because it provides one way to perform **unsupervised and semi-supervised learning**
- **Large amounts of unlabeled training data and relatively little labeled training data**.
- Training with supervised learning techniques on the labeled subset often results in severe overfitting.

Ktunotes.in

- **Semi-supervised learning offers the chance to resolve this overfitting problem by also learning from the unlabeled data**.
- we can learn good representations for the unlabeled data,
- **Greedy layer-wise unsupervised pretraining relies on a single-layer representation** learning algorithm such as an RBM, a single-layer autoencoder, etc.
- Each layer is pretrained using unsupervised learning, taking the output of the previous layer and producing as output a new representation of the data
- Greedy layer-wise pretraining is called **greedy** because it is a **greedy algorithm**, meaning that it optimizes each piece of the solution independently, one piece at a time, rather than jointly optimizing all pieces. It is called **layer-wise** because these independent pieces are the layers of the network.

KRMCOLLEGE.in

- **Greedy layer-wise pretraining proceeds one layer at a time**, training the k -th layer while keeping the previous ones fixed. In particular, the lower layers (which are trained first) are not adapted after the upper layers are introduced.
- It is called **unsupervised** because each layer is trained with an unsupervised representation learning algorithm.
- However it is also called **pretraining**, because it is supposed to be only a first step before a joint training algorithm is applied to **fine-tune** all the layers together.
- In the context of a supervised learning task, it can be viewed as a regularizer (in some experiments, pretraining decreases test error without decreasing training error) and a form of parameter initialization

- the entire two phase protocol that combines the **pretraining phase and a supervised learning phase**.
- The supervised learning phase may involve training a simple classifier on top of the features learned in the pretraining phase, or it may involve **supervised fine-tuning** of the entire network learned in the pretraining phase.

Krnotes.in

- **Distributed Representation--**Distributed representations of concepts—representations composed of many elements that can be set separately from each other—are one of the most important tools for representation learning
- **An important related concept that distinguishes a distributed representation from a symbolic one is that *generalization arises due to shared attributes*** between different concepts. As pure symbols, “ cat” and “ dog” are as far from each other as any other two symbols.
- For example, our distributed representation may contain entries such as “has_fur” or “number_of_legs” that have the same value for the embedding of both “ cat ” and “ dog.”

KRACKER.IO

- **Neural language models that operate on distributed representations of words generalize much better than other models that operate directly on one-hot representations of words**
- **Distributed representations induce a rich *similarity space*, in which semantically close concepts** (or inputs) are close in distance, a property that is absent from purely symbolic representations.

KNOWLEDGE.IO

RESEARCH AREAS- -BOLTZMANN MACHINES,

- Boltzmann machines were originally introduced as a general "**connectionist" approach to learning arbitrary probability distributions over binary vectors**
- We define the Boltzmann machine over a d-dimensional binary random vector $x \in \{0, 1\}^d$.
- The Boltzmann machine is **an energy-based model meaning** we define the **joint probability distribution** using an energy function:

$$P(\boldsymbol{x}) = \frac{\exp\left(-E(\boldsymbol{x})\right)}{Z}, \qquad (20.1)$$

Ktunotes.in KTU

where $E(x)$ is the energy function and Z is the partition function that ensures that $\sum_x P(x) = 1$. The energy function of the Boltzmann machine is given by

$$E(x) = -x^\top U x - b^\top x, \tag{20.2}$$

- where U is the "weight" matrix of model parameters and b is the vector of bias parameters.
- In the general setting of the Boltzmann machine, we are given a set of training examples, each of which are n-dimensional.
- **The Boltzmann machine becomes more powerful when not all the variables are observed**
- the **Boltzmann machine becomes a universal approximator of probability mass functions over discrete variables**

KRACLE.IO

- ***Boltzmann Machine Learning***
- Learning algorithms for Boltzmann machines are usually **based on maximum likelihood**.
- One interesting property of Boltzmann machines when trained with learning rules based on maximum likelihood is that the **update for a particular weight connecting two units** depends only the statistics of those two units, collected under different distributions

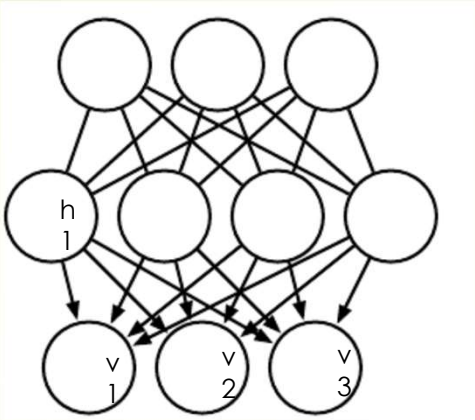
$$P_{\text{model}}(v) \text{ and } \hat{P}_{\text{data}}(v) P_{\text{model}}(h \mid v).$$

- the weight can be updated without knowing anything about the rest of the network or how those statistics were produced.
- **This means that the learning rule is "local," which makes Boltzmann machine learning somewhat biologically plausible**.
- if each neuron were a random variable in a Boltzmann machine, then the axons and dendrites connecting two random variables **could learn only by observing the firing pattern of the cells that they actually physically touch**
- In the positive phase, two units that frequently activate together have **their connection strengthened**. This is an example of a Hebbian learning rule--the mnemonic "fire together, wire together."

RESEARCH AREAS- DEEP BELIEF NETWORKS

- **Deep belief networks** (DBNs) were one of the first non-convolutional models to successfully admit training of deep architectures (Hinton *et al.*, 2006; Hinton, 2007b).
- **DBN is an algorithm for unsupervised probabilistic deep learning.**
- Deep belief networks are generative models with several layers of **latent variables**.
- The latent variables are **typically binary**, while the visible units may be binary or real. There are no intralayer connections
- every unit in each layer is connected to every unit in each neighboring layer, though it is possible to construct more sparsely connected DBNs. **The connections between the top two layers are undirected.**
- The connections between all other layers are directed, with the arrows pointed toward the layer that is closest to the data

RGANOTES.IN



- A DBN with l hidden layers contains l weight matrices: $W(1)$, . . . , $W(l)$. It also contains l+ 1 bias vectors: $b(0)$, . . . , $b(l)$, with $b(0)$ providing the biases for the visible layer. The probability distribution represented by the DBN is given by

$$P(\boldsymbol{h}^{(l)}, \boldsymbol{h}^{(l-1)}) \propto \exp\left(\boldsymbol{b}^{(l)\top}\boldsymbol{h}^{(l)} + \boldsymbol{b}^{(l-1)\top}\boldsymbol{h}^{(l-1)} + \boldsymbol{h}^{(l-1)\top}\boldsymbol{W}^{(l)}\boldsymbol{h}^{(l)}\right), \quad (20.17)$$

$$P(h_i^{(k)} = 1 \mid \boldsymbol{h}^{(k+1)}) = \sigma\left(b_i^{(k)} + \boldsymbol{W}_{:,i}^{(k+1)\top}\boldsymbol{h}^{(k+1)}\right) \forall i, \forall k \in 1, \ldots, l-2, \quad (20.18)$$

$$P(v_i = 1 \mid \boldsymbol{h}^{(1)}) = \sigma\left(b_i^{(0)} + \boldsymbol{W}_{:,i}^{(1)\top}\boldsymbol{h}^{(1)}\right) \forall i. \quad (20.19)$$

In the case of real-valued visible units, substitute

$$\mathrm{v} \sim \mathcal{N}\left(\boldsymbol{v}; \boldsymbol{b}^{(0)} + \boldsymbol{W}^{(1)\top}\boldsymbol{h}^{(1)}, \boldsymbol{\beta}^{-1}\right) \quad (20.20)$$

- with β diagonal for tractability
- Deep belief networks incur many of the problems associated with both directedmodels and undirected models
- **To train a deep belief network, one begins by training an RBM to maximize**

$$\mathbb{E}_{\mathbf{v} \sim p_{\text{data}}} \mathbb{E}_{\mathbf{h}^{(1)} \sim p^{(1)}(h^{(1)} \mid v)} \log p^{(2)}(h^{(1)}) \qquad (20.21)$$

- Ev~$p$data log $p(v)$ using contrastive divergence or stochastic maximum likelihood.
- The parameters of the RBM then define the parameters of the first layer of the DBN. Next, a second RBM is trained to approximately maximize

- **where *p*(1) is the probability distribution represented by the first RBM and *p*(2) is the probability distribution represented by the second RBM**.
- In other words, the second RBM is trained to model the distribution defined by sampling the hidden units of the first RBM, when the first RBM is driven by the data.
- This procedure can be repeated indefinitely, to add as many layers to the DBN as desired, with each new RBM modeling the samples of the previous one.
- **Each RBM defines another layer of the DBN**. This procedure can be justified as increasing a variational lower bound on the log-likelihood of the data under the DBN

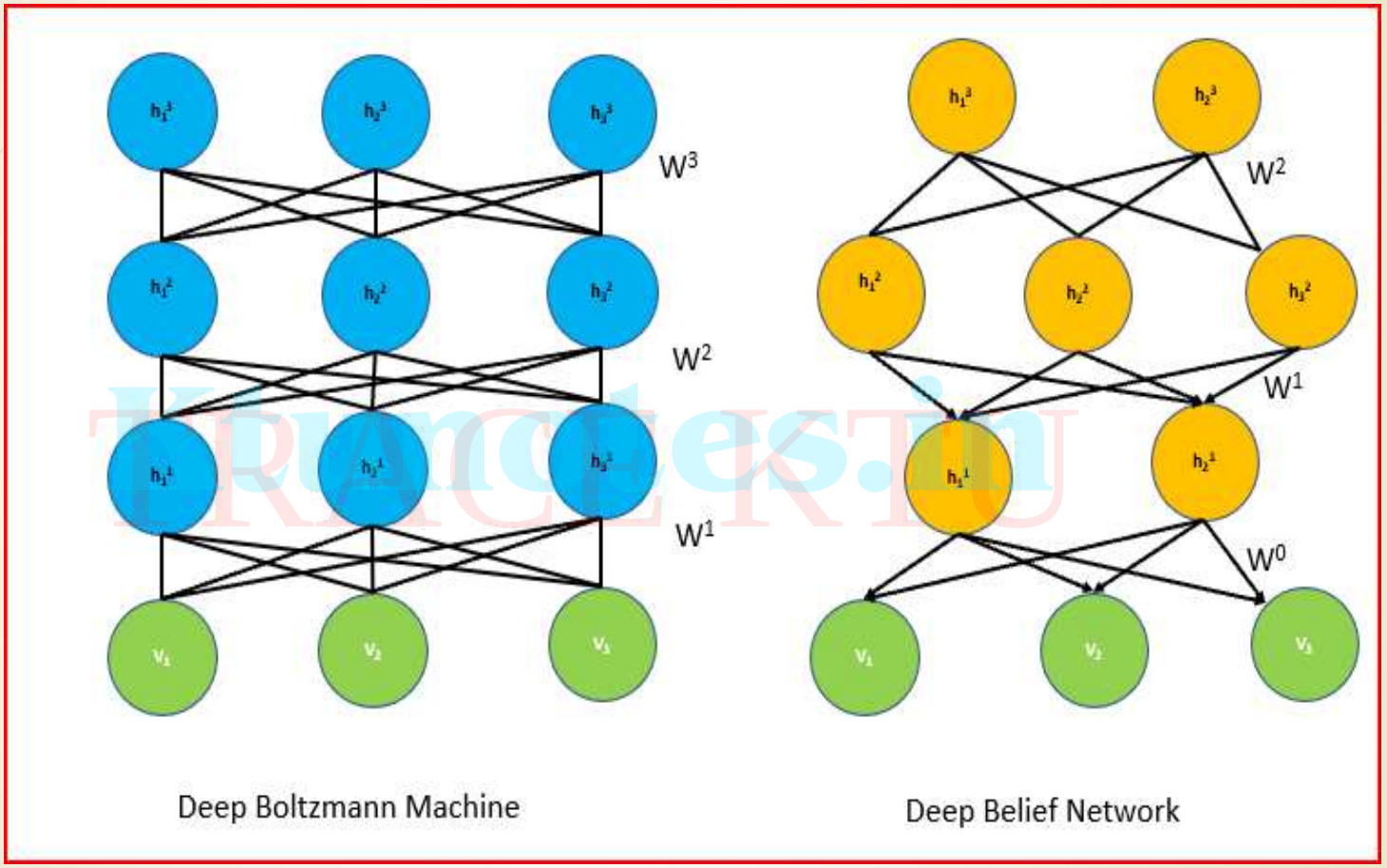
- The trained DBN may be used directly as a generative model, but most of the interest in DBNs arose from their ability to improve classification models.
- We can take the weights from the DBN and use them to define an MLP:

$$\boldsymbol{h}^{(1)} = \sigma\left(b^{(1)} + \boldsymbol{v}^{\top}\boldsymbol{W}^{(1)}\right). \tag{20.22}$$

$$\boldsymbol{h}^{(l)} = \sigma\left(b_i^{(l)} + \boldsymbol{h}^{(l-1)\top}\boldsymbol{W}^{(l)}\right) \forall l \in 2, \ldots, m, \tag{20.23}$$

Kraotes.in KRACE KTU

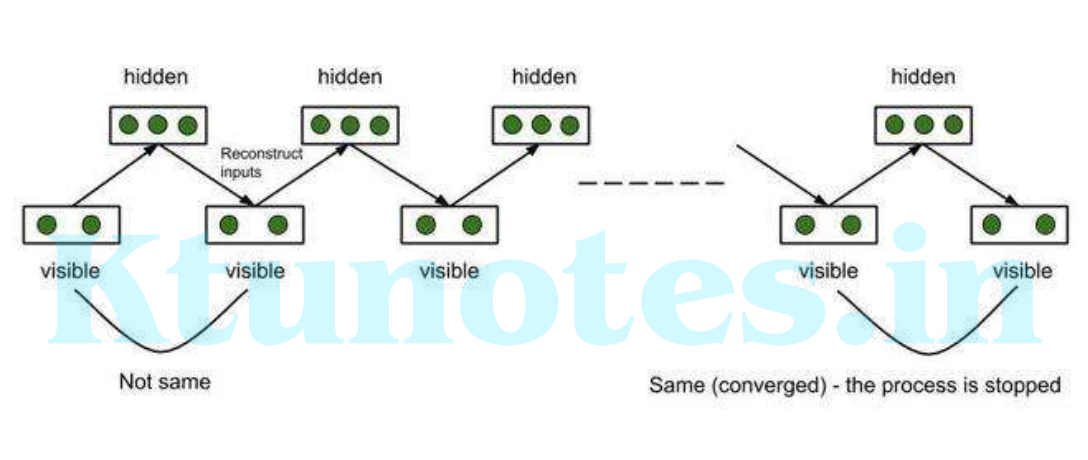
- The term "deep belief network" may also cause some confusion because the term "belief network" is sometimes used to refer to purely directed models, while **deep belief networks contain an undirected layer.** Deep belief networks also share the acronym DBN with dynamic Bayesian networks (Dean and Kanazawa, 1989), which are Bayesian networks for representing Markov chains.


W3
W2
W1
W2
W1
W0
Deep Boltzmann Machine
Deep Belief Network

- Deep Belief Network's operational pipeline is as follows:
- We'll use the Greedy learning algorithm to pre-train DBN. For learning the top-down generative weights-the greedy learning method that employs a layer-by-layer approach. These **generative weights determine the relationship between variables in one layer and variables in the layer above**.
- On the top two hidden layers, we run numerous steps of **Gibbs sampling in DBN**. The top two hidden layers define the RBM thus, this stage is effectively extracting a sample from it.
- Then generate a sample from the visible units using a single pass of ancestral sampling through the rest of the model.
- We'll use a single bottom-up pass to infer the values of the latent variables in each layer. In the bottom layer, greedy pretraining begins with an observed data vector. It then oppositely fine-tunes the generative weights.

- **Contrastive Divergence:**
- **RBM** adjusts its weights by this method. Using some randomly assigned initial weights,
- RBM calculates the hidden nodes, which in turn use the same weights to reconstruct the input nodes.
- Each hidden node is constructed from all the visible nodes and each visible node is reconstructed from all the hidden node and hence, the input is different from the reconstructed input, though the weights are the same.
- The process continues until the reconstructed input matches the previous input. The process is said to be converged at this stage. This entire procedure is known as **Gibbs Sampling**.

KRACKEN.IO



Ktunotes.in

Fig:-Gibbs sampling

- calculate the binary states of the hidden layers in the positive phase by **computing the probabilities of weights and visible units**.
- It is known as the **positive phase since it enhances the likelihood of the training data set.**
- The negative phase **reduces the likelihood of the model producing samples.**
- To train a complete Deep Belief Network,
- Several RBMs together make a Deep Belief Networks

Ktunotes.in

- **Applications**
- We employ deep belief networks in place of deep feedforward networks or even convolutional neural networks
- They have the benefit of being less computationally costly. computational complexity grows linearly with the number of layers, rather than exponentially as with feedforward neural networks) and is less susceptible to the vanishing gradients problem.
- Applications of DBN are as follows:

- Recognition of images.
- Sequences of video.
- Data on mocap.
- Speech recognition.

Ktunotes.in